Technical Report - Efficient Time Series Transformer Architecture for Generalised Two-Wheeler Driving Event Recognition

Arihant Jain, Geethika Palla, Deepak Gangadharan
Computer Systems Group, IIIT Hyderabad, India
{arihant.j, geethika.palla}@research.iiit.ac.in, deepak.g@iiit.ac.in

***Abstract*—Reliable recognition of motorcycle driving events is important for rider safety and intelligent mobility applications. This work addresses the limited generalisability of earlier studies by introducing a more diverse real-world time-series dataset collected using 3-axis accelerometer and gyroscope sensors from both male and female riders, recorded during day and night riding. The dataset captures wider variations in riding behaviour and sensor dynamics, making event recognition more realistic and challenging. To model this increased variability, we focus on a Time-Series Transformer (TST) architecture that can capture long-range temporal dependencies while remaining efficient for embedded deployment. The proposed approach is evaluated on edge hardware, where inference latency, memory usage, and model size are analysed. Experimental results show that the TST achieves strong and consistent class-wise performance while maintaining low computational overhead, making it well suited for real-time on-device two-wheeler event recognition.**

***Index Terms*—temporal convolutional networks, transformer-based models, two-wheeler event recognition**

I. Introduction

Two-wheelers are widely used in many developing and densely populated regions, but they are also associated with a large proportion of road traffic fatalities. According to the World Health Organization, motorcyclists and other powered two- and three-wheeler riders account for approximately 30% of global road traffic deaths, increasing to 46% in the South-East Asia region [1]. Motorcycle riding often involves sudden manoeuvres and strong rider-dependent behaviour, which are key factors in crash analysis studies [2].

Automatic recognition of riding events such as turns, braking, bumps, and stops can support safety monitoring, rider-assistance systems, and intelligent mobility applications. For practical deployment, these systems must perform reliably under real-world riding conditions.

Driving-event recognition has been extensively studied for four-wheelers since the mid-2000s, evolving from probabilistic models to machine-learning and deep-learning approaches [3], [6]. In contrast, research on two-wheelers has gained attention only recently, enabled by advances in inertial sensing and embedded platforms [4], [5]. However, many existing studies rely on datasets collected from a limited number of riders or under relatively controlled conditions.

A major limitation of such datasets is the lack of diversity in riding behaviour. Riding patterns vary across individuals, between male and female riders, and across different times of day. These variations produce sensor patterns that are often underrepresented in narrowly collected datasets, leading to models that perform well during testing but struggle to generalise in real-world scenarios.

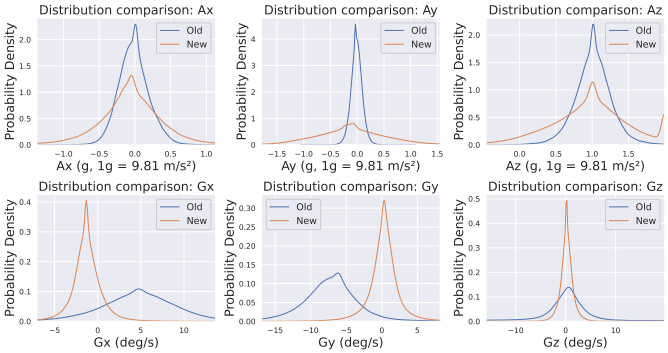

Fig. 1: Comparison of accelerometer and gyroscope signal distributions between the dataset used in DATE 2023 [4] (old) and the proposed dataset (new).

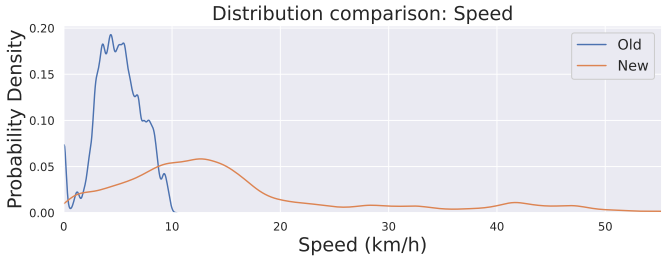

Fig. 2: Comparison of speed distributions between the dataset used in DATE 2023 [4] (old) and the proposed dataset (new).

This limitation is illustrated in Fig. 1 and Fig. 2, which compare the feature distributions of the dataset used in DATE 2023 [4] with the dataset collected in this work. In both figures, the blue curves correspond to the earlier dataset, while the orange curves represent the proposed dataset. The proposed dataset exhibits wider distributions in accelerometer and gyroscope signals as well as a broader speed range, indicating greater variability in riding dynamics and more realistic riding conditions.

To address these limitations, we collect a new multivariate time-series dataset consisting of tri-axial accelerometer and gyroscope measurements together with GPS-derived speed.

The dataset includes recordings from both male and female riders and covers rides performed during both day and night, resulting in broader sensor distributions and more complex temporal patterns.

Using this dataset, we propose a Time-Series Transformer (TST) model for generalised two-wheeler driving-event recognition. The model captures long-range temporal relationships across sensor signals and is evaluated against several state-of-the-art approaches, including LSTM, Bi-LSTM with and without attention, Temporal Convolutional Networks (TCN), and TCN with Chunked Attention (TCN-CA). All models are tuned using systematic hyperparameter optimisation to ensure a fair comparison.

Finally, to evaluate practical feasibility, all models are deployed on a Raspberry Pi edge platform, where inference latency and memory usage are measured to assess real-time deployment capability.

The main contributions of this work are summarised as follows:

- A new and more diverse two-wheeler time-series dataset including male and female riders and data collected during both day and night.
- A transformer-based approach using a Time-Series Transformer (TST) for generalised two-wheeler driving-event recognition, evaluated against existing recurrent and convolutional state-of-the-art models.
- An evaluation of on-device feasibility through deployment and benchmarking on a Raspberry Pi platform.

II. Related Work

Driving-event recognition aims to detect riding manoeuvres from sensor data. While many studies focus on four-wheelers, fewer works address two-wheelers, where rider-dependent behaviour and dynamic motion make the problem more challenging.

A. Two-Wheeler Driving Event Recognition

Most existing research on motorcycle driving-event recognition relies on inertial sensor data collected from either dedicated sensor modules or smartphones. Raheel *et al.* [5] studied motorbike activity recognition using smartphone motion sensors and traditional supervised learning methods. Although smartphones offer an easy sensing solution, variations in device placement and orientation can lead to inconsistent measurements.

A closely related work is the study by Goparaju *et al.* [4], which introduced a two-wheeler dataset collected using accelerometer and gyroscope sensors and evaluated LSTM and Bi-LSTM models for event recognition. Their results showed that recurrent neural networks can effectively classify riding manoeuvres. However, the dataset was collected under relatively structured conditions with limited rider diversity, which may restrict generalisation to real-world riding scenarios.

Overall, existing two-wheeler studies demonstrate the feasibility of driving-event recognition but are often limited by dataset size, rider diversity, or the range of riding conditions considered.

B. Temporal Models for Sensor-Based Event Recognition

Beyond vehicle-specific studies, several works have investigated temporal models for general time-series classification using sensor data. Temporal Convolutional Networks (TCNs) were shown by Bai *et al.* [12] to perform competitively against recurrent models by capturing temporal dependencies using causal convolutions.

Transformer-based architectures have also been adapted for time-series modelling. Zerveas *et al.* [13] proposed the Time-Series Transformer, demonstrating that self-attention can effectively model long-range temporal dependencies in multivariate sensor data. These advances motivate the evaluation of convolutional and transformer-based models for two-wheeler event recognition.

Several studies have also explored deep-learning models for sensor-based time-series classification beyond vehicle-specific applications. Zeng *et al.* [7] and Ordóñez and Roggen [8] demonstrated that convolutional and recurrent architectures can effectively learn motion patterns from accelerometer and gyroscope data. Wang *et al.* [9] further showed that deep neural networks can outperform traditional feature-based methods for time-series classification.

C. Context from Four-Wheeler Studies

Driving-event recognition has also been explored for four-wheelers using probabilistic and machine-learning approaches. For example, Mitrović [3] proposed a Hidden Markov Model–based method for recognising car driving events, and Ferreira *et al.* [6] studied driver behaviour profiling using smartphone sensors. These works provide useful methodological insights but are not directly applicable to two-wheelers due to the higher sensor variance and rider-dependent dynamics of motorcycles.

D. Summary and Research Gap

In summary, while several studies have explored two-wheeler driving-event recognition, existing datasets and evaluations often lack sufficient diversity in terms of riders, riding behaviour, and operating conditions. Moreover, recent advances in temporal modelling, such as TCNs and transformer-based architectures, have not been extensively evaluated on diverse two-wheeler datasets. This gap motivates our work, which introduces a more representative dataset and a systematic comparison of recurrent, convolutional, and transformer-based models for two-wheeler driving-event recognition.

III. Model and Training Methodology

This section describes the temporal models evaluated in this work, the training setup, and the hyperparameter optimisation strategy. All models operate on multivariate time-series sensor data and are trained under identical conditions to enable fair comparison.

A. Proposed Temporal Driving Event Recognition Models

1) Time-Series Transformer (TST)

Fig. 3 illustrates the architecture of the proposed Time-Series Transformer (TST), which builds on the transformer

encoder framework introduced by Vaswani *et al.* [11] and later adapted for time-series modelling by Zerveas *et al.* [13]. The model operates on fixed-length multivariate inertial sensor sequences for driving-event recognition.

The input to the model is a sensor window

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T] \in \mathbb{R}^{T \times Z}, \tag{1}$$

where T denotes the number of time steps in the window and Z=7 represents the sensor channels (accelerometer, gyroscope, and speed).

The raw sequence is first processed by a convolutional feature embedding module to capture short-range temporal patterns and transform the input signals into a latent representation. Two one-dimensional convolutional layers are applied along the temporal dimension:

$$\mathbf{E}^{(1)} = \sigma(\text{Conv1D}_{k_1}(\mathbf{X})), \tag{2}$$

$$\mathbf{E}^{(2)} = \sigma(\text{Conv1D}_{k_2}(\mathbf{E}^{(1)})), \tag{3}$$

where Conv1D_k denotes a one-dimensional convolution with kernel size k, $\sigma(\cdot)$ is the activation function, and the second convolution produces the embedding dimension d_{model}. The result is linearly projected into a latent space of dimension d_{model} using a learnable weight matrix W_e and bias b_e:

$$\mathbf{F} = E^{(2)} W_e + b_e, W_e \in \mathbb{R}^{d_{model} \times d_{model}} \text{ and } b_e \in \mathbb{R}^{d_{model}} \tag{4}$$

serves as the input to the transformer encoder.

To encode temporal order, positional embeddings $\mathbf{P} \in \mathbb{R}^{T \times d_{\text{model}}}$ are added to the embedded features:

$$\mathbf{H}^{(0)} = \mathbf{F} + \mathbf{P}. \tag{5}$$

Both learned and sinusoidal positional encodings are considered, with the optimal choice selected through hyperparameter optimisation.

The sequence is then processed by a stack of L transformer encoder layers. In each layer ℓ, layer normalization is first applied to the input feature vector $\mathbf{h}$ at each time step to stabilise training by normalising across the feature dimension:

$$\text{LN}(\mathbf{h}) = \gamma \frac{\mathbf{h} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \tag{6}$$

where μ and σ^2 denote the mean and variance of $\mathbf{h}$, and γ and β are learnable scale and shift parameters.

Self-attention is then computed by projecting the output of the previous encoder layer, $\mathbf{H}^{(\ell-1)}$, into query, key, and value representations:

$$\mathbf{Q} = \mathbf{H}^{(\ell-1)} \mathbf{W}_Q, \quad \mathbf{K} = \mathbf{H}^{(\ell-1)} \mathbf{W}_K, \quad \mathbf{V} = \mathbf{H}^{(\ell-1)} \mathbf{W}_V, \tag{7}$$

where $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are learnable projection matrices. Scaled dot-product attention is then computed as

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \tag{8}$$

where d_k is the dimensionality of each attention head used for scaling. Different attention variants, including multi-head, additive, scaled dot-product, and window-based attention, are explored during tuning, and the best-performing configuration is selected.

The attention output is combined with a residual connection and passed through a second layer normalization before entering a position-wise feed-forward network:

$$\text{FFN}(\mathbf{h}) = \mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2, \tag{9}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_\text{model} \times ff_\text{dim}}$ projects the features into an intermediate feed-forward space of dimension ff_dim, $\mathbf{W}_2$ maps them back to d_model, and $\mathbf{b}_1$, $\mathbf{b}_2$ are bias terms. Dropout is applied after both attention and feed-forward blocks.

After the encoder stack, temporal features are aggregated using global average pooling:

$$\mathbf{z} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{H}_t^{(L)}, \tag{10}$$

where $\mathbf{H}_t^{(L)}$ denotes the output of the final encoder layer at time step t. Finally, a fully connected classification layer with weight matrix $\mathbf{W}_c$ and bias $\mathbf{b}_c$, followed by softmax activation, produces class probabilities over K driving-event classes:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_c \mathbf{z} + \mathbf{b}_c). \tag{11}$$

Architectural Differences from the Original TST: While our model is based on the Time-Series Transformer framework of Zerveas *et al.* [13], several modifications are introduced. First, a convolutional feature embedding module is applied before the transformer to capture short-term temporal patterns in IMU signals, whereas the original TST primarily relies on direct linear projection. Second, sliding-window multi-head self-attention is used instead of full self-attention, reducing computational complexity and improving efficiency for real-time edge deployment. Third, the encoder adopts a pre-layer-normalisation structure, which improves training stability compared to the post-normalisation design used in the original formulation.

In our experiments, the best-performing TST uses a convolutional embedding module consisting of two 1D convolutional layers with 64 filters and kernel size 9 in the first layer, followed by d_{model}=16 filters with kernel size 3 in the second layer, both using ReLU activation and even padding to the left/right or up/down of the input. The transformer uses sinusoidal positional encoding, two attention heads, a feed-forward dimension of ff_{dim}=64, sliding-window self-attention with window size 7, dropout of 0.1, and a single encoder layer. The model is trained using the AdamW optimizer with a learning rate of 3×10^{-4}, weight decay of 1×10^{-4}, and gradient clipping with a norm of 1.0. Label smoothing with a factor of 0.1 is applied in the categorical cross-entropy loss to improve generalisation.

This formulation enables the TST to capture both local and long-range temporal dependencies while remaining compact and suitable for efficient on-device deployment.

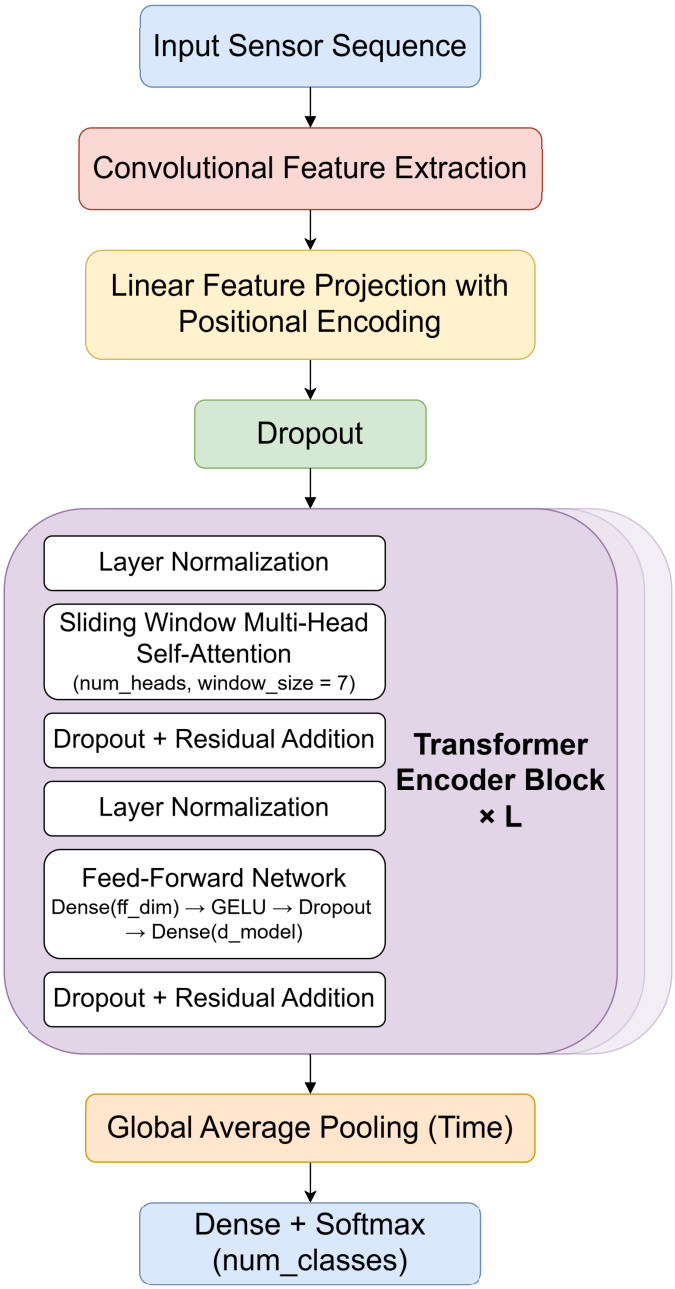

Fig. 3: TST Architecture

2) TCN with Chunked Attention (TCN-CA)

The TCN with Chunked Attention [14] combines convolution-based temporal modelling with local self-attention. The model first applies a Temporal Convolutional Network (TCN) to the input sensor sequence. The TCN uses stacked one-dimensional causal convolutions with increasing dilation factors, which allow it to capture both short-term and long-term temporal patterns while preserving sequence order. Skip connections and batch normalisation are used to improve training stability and convergence.

The TCN output is then processed by a chunked multi-head self-attention layer. The sequence is divided into fixed-length, non-overlapping chunks, and self-attention is applied

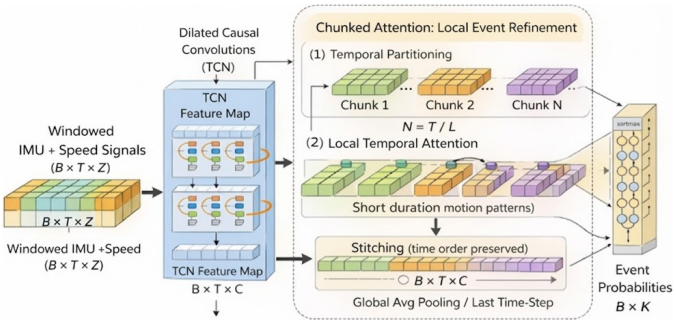

Fig. 4: TCN-CA Architecture

Legend: B batch size, T time steps, $Z(=7)$ input features per step, C TCN channels, L chunk length, $K(=)5$ output classes.

independently within each chunk. This allows the model to focus on local temporal relationships without attending to the full sequence, reducing computational cost and limiting the effect of distant noise. The attended features are finally aggregated using global average pooling and passed through fully connected layers to produce the event prediction.

B. Hyperparameter Optimisation

All models are tuned using grid-search-based hyperparameter optimisation, with configurations selected based on validation performance.

For the Time-Series Transformer (TST), the search space includes the embedding dimension, number of encoder layers and attention heads, feed-forward dimension, dropout rate, positional encoding type, batch size, learning rate, weight decay, and input sequence length.

For the TCN with Chunked Attention (TCN-CA), both convolutional and attention parameters are tuned, including the number of filters, kernel size, dilation schedule, activation function, dropout rate, batch size, as well as chunk size and attention head dimensions.

C. Training Configuration

All models are trained using the Adam optimizer and categorical cross-entropy loss:

$$\mathcal{L} = \mathrm{CE}(\mathbf{y}, \hat{\mathbf{y}}). \tag{12}$$

Model performance is monitored using classification accuracy. Early stopping is applied based on validation loss with a patience of ten epochs. Training is stopped when validation loss no longer improves. Sequence length, batch size and number of epochs for each model are selected based on the outcomes of the grid search.

Different sequence lengths were evaluated individually for each model during hyperparameter optimisation. A window length of $T = 156$ consistently produced the best validation performance for all architectures and was therefore used for all experiments. No synthetic data augmentation was applied; all models were trained using only the original recorded sensor data.

IV. Dataset Description

This work introduces a new multivariate time-series dataset of motorcycle dynamics collected under natural riding conditions. The dataset was designed to be more representative and generalisable than prior collections by incorporating rider diversity, variations in riding style, and recordings across different times of day.

A. Hardware Setup and Sensor Placement

Data were recorded using an MPU - 6500 inertial measurement unit (IMU), which provides tri-axial acceleration and angular velocity measurements. The accelerometer outputs acceleration in units of g, where $1\,g = 9.81\,\mathrm{m/s^2}$, corresponding to the acceleration due to Earth's gravity. A GY - GPS6MV2 module was used to obtain vehicle speed. Both sensors were interfaced with a Raspberry Pi 4 Model B, which logged synchronised measurements at 104 Hz.

The sensing node was positioned at the footrest centre to ensure stable mounting. This placement provides a consistent reference frame across riders.

B. Riders and Data Collection Characteristics

Recordings were collected from nine riders: four female and five male participants. Sessions were conducted during both daytime and nighttime operation to capture natural temporal variability. The final dataset comprises 4,66,315 samples, with the following distribution:

- **Male:** 2,01,312 samples,
- **Female:** 2,65,003 samples,
- **Day:** 2,50,700 samples,
- **Night:** 2,15,615 samples.

Each sample contains a 7-dimensional feature vector: three-axis acceleration, three-axis gyroscope readings, and speed.

C. Event Classes

Five driving events commonly encountered in everyday motorcycle operation were annotated:

- **STRAIGHT:** 3,34,387 samples,
- **BUMP:** 49,779 samples,
- **STOP:** 31,520 samples,
- **LEFT:** 27,796 samples,
- **RIGHT:** 22,105 samples.

The distribution reflects typical riding behaviour, with straight-line motion dominating and manoeuvres such as turns appearing less frequently.

D. Male–Female Riding Behaviour Differences

To examine whether rider characteristics influence the recorded sensor patterns, we analysed and compared the feature distributions of male and female riders. Fig. 5 shows the distribution of accelerometer and gyroscope signals for both groups. The kernel density plots indicate small but noticeable shifts across several axes. These differences are subtle and are likely due to natural variations in posture, balance, and how riders handle the vehicle, rather than any major change in riding behaviour.

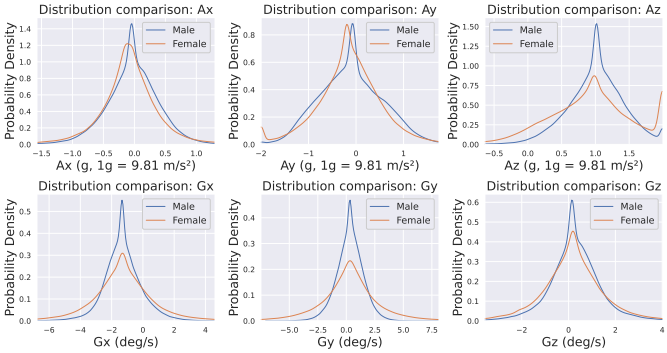

Fig. 5: Distribution of accelerometer and gyroscope signals (Ax, Ay, Az, Gx, Gy, Gz) for male (blue) and female (orange) participants.

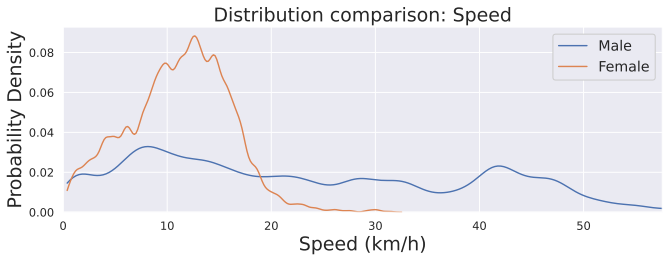

Fig. 6: Speed distribution male (blue) and female (orange) participants.

The Population Stability Index (PSI) values further quantify these differences. For most IMU features, the PSI remains low, with values of 0.040 for Ax, 0.051 for Ay, and 0.062 for Gz, indicating very similar distributions across male and female riders. Slightly higher but still moderate shifts are observed for Az (0.230), Gx (0.190), and Gy (0.328), reflecting subtle differences in vertical motion and rotational behaviour.

In contrast, speed shows a much larger PSI value of 4.333, indicating a clear distributional difference between male and female riders. As shown in Fig. 6, the speed distribution differs more strongly between the two groups, suggesting variations in preferred cruising behaviour rather than differences in manoeuvre execution. Together, these results show that while core riding dynamics remain largely similar, speed variation and small sensor-level differences add realistic diversity that is important for building generalisable event recognition models.

E. Train–Validation–Test Split

To prevent temporal leakage and preserve the sequential nature of the data, the dataset is divided chronologically. The first 65% of the dataset is used for training, while the remaining 35% is further divided chronologically into validation (17.5%) and test (17.5%) subsets. This ensures that the model is always evaluated on data occurring later in time than the training portion, better reflecting deployment scenarios in which future behaviour must be inferred from past observations.

F. Summary

The combination of diverse riders, day–night variation, stable sensor placement, and naturally imbalanced event frequencies

results in a dataset that captures a wide range of realistic motorcycle dynamics. This increased variability makes the dataset more challenging and more representative than earlier collections, providing a stronger benchmark for evaluating temporal deep-learning models for two-wheeler event recognition.

V. Results & Analysis

This section discusses the performance of the seven temporal models evaluated on the proposed dataset. All models were trained and tested using the same chronological train–validation–test split. Overall accuracy, Macro F1 score, and class-wise accuracy are used to analyse model behaviour across different riding events.

A. Performance of Recurrent and Attention-based Models

Table I summarises the performance of all evaluated models using accuracy, Macro F1, and class-wise accuracy for each driving event.

Among the recurrent baselines, LSTM and Bi-LSTM achieve similar performance, with overall accuracies of 72.08% and 72.66%, respectively. Introducing attention leads to noticeable improvements for both architectures. LSTM with Attention increases the accuracy to 73.80%, while Bi-LSTM with Attention reaches 74.57%. These results indicate that attention helps the models emphasise informative temporal segments within the sequence.

Across different riding events, recurrent models perform well on the dominant *Straight* class but show weaker performance on manoeuvre-related events such as turns and bumps. These events involve shorter and more complex motion patterns, which are more difficult for recurrent architectures to capture reliably.

B. Comparison of Temporal Architectures

Compared to the recurrent models, the TCN with Chunked Attention achieves stronger performance, reaching an overall accuracy of 75.14%. By combining convolution-based temporal modelling with localized chunked attention, the model focuses on short-term motion patterns while reducing the influence of noise from longer sequences.

The Time-Series Transformer (TST) achieves the same overall accuracy of 75.14% but delivers the strongest overall evaluation metric with the highest Macro F1 score of 72.84. A closer examination of the class-wise results shows that the TST outperforms TCN-CA across most manoeuvre-related events, including *Stop*, *Right Turn*, *Left Turn*, and *Bump*. The only class where TCN-CA performs better is *Straight*, which is also the most dominant class in the dataset.

Overall, the results show that the TST recognises the more challenging and less frequent driving events more effectively, resulting in the strongest overall performance among the evaluated models.

VI. Edge Deployment on Raspberry Pi

To assess real-time feasibility and resource efficiency, all seven temporal models were deployed on a Raspberry Pi 4 Model B (4 GB RAM). The evaluation consists of two components: (i) offline benchmarking of computational characteristics and (ii) live inference using streaming sensor data. Together, these experiments assess both the practical and deployment suitability of each architecture for on-device execution.

A. Model Reconstruction on Device

After training, each model's architecture and weights were moved to the Raspberry Pi. The complete TensorFlow Keras models were rebuilt on the device and the weights were loaded as they were, without quantization. For models with custom components like positional embeddings, transformer encoder layers, or attention modules, the required classes were registered through the Keras serialization system so that the models could be loaded properly. This ensures that the model running on the Pi matches the one obtained after training.

B. Benchmarking Methodology

We benchmark all models under identical conditions to measure computational load and efficiency. The following metrics are reported:

- **Trainable parameters:** Obtained from model summaries, indicating memory requirements and architectural complexity.
- **Inference latency:** Timed inference runs were performed after a few warm-up runs. For each run, total batch inference time was recorded and normalised to obtain per-sequence latency.
- **Memory usage:** During each timed run, the process' resident set size (RSS) was periodically sampled to measure memory consumption. Peak RSS was recorded for every run.

Each model underwent 5 warm-up iterations followed by $N_{\text{runs}} = 10$ timed runs. Table II reports test accuracy, number of trainable parameters, average per-sequence latency, average peak memory usage, and maximum observed peak memory for all models. These metrics allow a direct comparison between recognition performance and deployment efficiency.

C. Deployment Performance Comparison

The Time-Series Transformer (TST) achieves the highest accuracy of 75.14% while also maintaining the smallest number of trainable parameters (10,821). Despite its compact architecture, the model delivers stable inference latency (3.614 ms per sequence) and controlled memory usage, making it well suited for real-time deployment on edge hardware.

In contrast, the TCN with Chunked Attention model has the largest parameter count (1.67M), resulting in substantially higher memory consumption and the longest inference time per sequence (31.807 ms). Although its recognition accuracy is competitive, the significantly higher computational cost makes it less practical for resource-constrained edge devices.

Traditional recurrent models such as LSTM and Bi-LSTM provide relatively fast inference, with the LSTM with Attention variant achieving the lowest latency (2.589 ms). However, their prediction accuracy remains noticeably lower than that of the TST. Attention-based recurrent variants slightly improve

TABLE I: Performance comparison of different models (all values in %).

Model	Accuracy	Macro F1	Straight (SL)	Stop (ST)	Right (RT)	Left (LT)	Bump (SB)
LSTM [10]	72.08	69.34	83.93	74.19	56.25	65.52	55.17
LSTM with Attention	73.80	71.57	83.93	83.87	62.50	70.69	50.86
Bi-LSTM	72.66	70.03	82.14	78.49	59.38	62.07	58.62
Bi-LSTM with Attention	74.57	71.84	81.25	82.80	59.38	72.41	60.34
TCN	73.04	69.73	86.16	88.17	56.25	72.41	40.52
TCN with Chunked Attention	75.14	72.32	91.52	76.34	59.38	62.07	53.45
TST	**75.14**	**72.84**	**85.27**	**78.49**	**65.62**	**62.07**	**62.07**

TABLE II: Deployment performance of selected models on Raspberry Pi 4 Model B

Model	Accuracy (%)	Trainable Params	Time / Seq (ms)	Avg Peak Mem (MB)	Max Peak Mem (MB)
LSTM	72.08	21,237	3.070	516.68	518.14
LSTM with Attention	73.80	6,485	2.589	512.17	513.28
Bi-LSTM	72.66	46,605	3.968	470.85	473.50
Bi-LSTM with Attention	74.57	73,005	5.760	547.85	553.99
TCN	73.04	5,66,405	17.651	630.65	649.97
TCN with Chunked Attention	75.14	16,77,381	31.807	748.07	774.88
TST	**75.14**	**10,821**	**3.614**	**565.47**	**572.67**

recognition performance but introduce additional parameters and memory overhead.

D. Live Inference Framework

In addition to offline benchmarking, a real-time inference pipeline was implemented using streaming sensor data. The system continuously monitors a CSV file generated by the data collection process and extracts new sensor samples as they arrive. These samples are stored in a FIFO buffer, from which fixed-length windows are formed for prediction.

1) Windowing and Batch Formation

Non-overlapping windows of length 156 samples are formed, consistent with the configuration used during training of the model. As soon as a window is available, its absolute starting index is recorded, and windows are accumulated into batches. When the batch size reaches the required number of windows or when no new samples have arrived for a short idle duration, the batch is forwarded to the model for inference.

2) Prediction and Logging

For each window, the model outputs class probabilities. The following information is recorded in an output CSV file:

- prediction timestamp,
- window start and end indices,
- predicted class index and label,
- class probability distribution,
- batch inference time in milliseconds.

This produces a chronological stream of driving-event predictions suitable for later analysis or real-time rider-assistance applications.

3) Real-Time Performance

During live deployment, all evaluated models were capable of processing incoming data in real time without lag, operating faster than the 104 Hz sensor sampling rate. Among them, the TST maintained stable inference latency while delivering the highest recognition accuracy, making it particularly suitable for continuous on-device inference. While simpler recurrent models achieved slightly lower latency, the improved predictive performance of the TST makes it a stronger candidate for practical rider-assistance systems deployed on embedded hardware such as the Raspberry Pi.

VII. Conclusion

This work studied two-wheeler driving-event recognition using a diverse real-world dataset including multiple riders, gender variation, and day–night riding conditions. Several temporal models were evaluated to analyse their ability to handle real-world variability.

The results show that modern temporal architectures outperform traditional recurrent models. In particular, the Time-Series Transformer (TST) achieved the strongest overall performance, matching the highest accuracy while obtaining the best Macro F1 and MCC scores and superior recognition across most event classes. With its compact architecture and stable inference latency, the TST is well suited for real-time deployment on resource-constrained edge devices.

References

[1] Global status report on road safety 2023. Geneva: World Health Organization; 2023. Licence: CC BY-NC-SA 3.0 IGO. Available: https://www.who.int/publications/i/item/9789240086517

[2] Sultan, Z.; Ngadiman, N.I.; Kadir, F.D.A.; Roslan, N.F.; Moeinaddini, M. "Factor Analysis of Motorcycle Crashes in Malaysia." Available: https://www.planningmalaysia.org/index.php/pmj/article/view/SI-4-11

[3] D. Mitrovic, "Reliable method for driving events recognition" in IEEE Transactions on Intelligent Transportation Systems, vol. 6, no. 2, pp. 198-205, June 2005, doi: 10.1109/TITS.2005.848367.

[4] S. U. N. Goparaju, L. Lakshmanan, A. Navnit, R. Biju, L. B., D. Gangadharan, and A. M. Hussain, "Time Series-based Driving Event Recognition for Two Wheelers," in *Proc. Design, Automation & Test in Europe Conf. (DATE)*, 2023.

[5] A. Raheel, A. Arsalan, S. H. Noorani, S. Khan, M. Ehatisham-Ul-Haq, and Z. Ali, "Motorbike Driving Activity Recognition Using Smartphone Motion Sensors," in *Proc. 2019 22nd IEEE Int. Multitopic Conf. (INMIC)*, 2019.

[6] J. J. Ferreira Júnior, E. Carvalho, B. V. Ferreira, C. de Souza, Y. Suhara, A. Pentland, and G. Pessin "Driver behavior profiling: An investigation with different smartphone sensors and machine learning," *PLOS ONE*, vol. 12, no. 4, e0174959, 2017.

[7] M. Zeng, L. T. Nguyen, B. Yu, O. J. Mengshoel, J. Zhu, P. Wu, and J. Zhang, "Convolutional Neural Networks for Human Activity Recognition Using Mobile Sensors," in *Proc. 6th Int. Conf. Mobile Computing, Applications and Services (MobiCASE)*, Austin, TX, USA, 2014, pp. 197–205.

[8] F. J. Ordóñez and D. Roggen, "Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition," *Sensors*, vol. 16, no. 1, p. 115, 2016.

[9] Z. Wang, W. Yan, and T. Oates, "Time Series Classification from Scratch with Deep Neural Networks," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, Anchorage, AK, USA, 2017, pp. 1578–1585.

[10] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Proc. 31st Int. Conf. Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 5998–6008.

[12] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[13] G. Zerveas, S. Jay, S. P. Roberts, M. Bagnell, and A. S. Stolcke, "A Transformer-Based Framework for Multivariate Time Series Representation Learning," in *Proc. 27th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*, 2021, pp. 2114–2124.

[14] A. Terzić, M. Hersche, G. Karunaratne, L. Benini, A. Sebastian, and A. Rahimi, "TCNCA: Temporal Convolution Network with Chunked Attention for Scalable Sequence Processing," *arXiv preprint arXiv:2205.00456*, 2022.

[15] Unit8, "Temporal Convolutional Networks and Forecasting," Unit8 Research Blog, 2019. Available: Temporal Convolutional Networks and Forecasting

[16] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.